

Ejecución de Instrucciones

22026

Juan Camilo Acevedo Bedoya | Facultad de Ingenierías

Arquitectura Básica del MIPS

La arquitectura MIPS se basa en el diseño RISC (Reduced Instruction Set Computing), que se caracteriza por un conjunto de instrucciones reducido y una ejecución más rápida. Los componentes principales incluyen la CPU, la memoria y los periféricos de E/S. La CPU gestiona la ejecución de instrucciones, mientras que la memoria almacena tanto las instrucciones como los datos necesarios.

Dentro de la CPU, los registros de propósito general juegan un papel crucial. Estos registros almacenan operandos y resultados intermedios, facilitando operaciones rápidas y eficientes. El diseño RISC del MIPS promueve la simplicidad y la eficiencia, lo que lo convierte en una excelente plataforma para el aprendizaje de la arquitectura de computadores.

Diseño RISC

Conjunto de instrucciones reducido para mayor eficiencia.

CPU

Unidad central de procesamiento que gestiona la ejecución de instrucciones.

Registros

Almacenamiento rápido de operandos y resultados intermedios.

Formato de Instrucciones MIPS

Las instrucciones MIPS tienen un tamaño fijo de 32 bits. Esta uniformidad facilita la decodificación y ejecución de las instrucciones. Existen tres tipos principales de instrucciones: R (Register), I (Immediate) y J (Jump). Cada tipo tiene una estructura diferente para adaptarse a distintas operaciones.

Los campos de las instrucciones varían según el tipo, pero generalmente incluyen el código de operación (opcode), los registros fuente (rs, rt), el registro destino (rd), el desplazamiento (immediate) y la dirección de salto. Comprender estos campos es crucial para interpretar y escribir código ensamblador MIPS.

Tipo R

Operaciones entre registros.

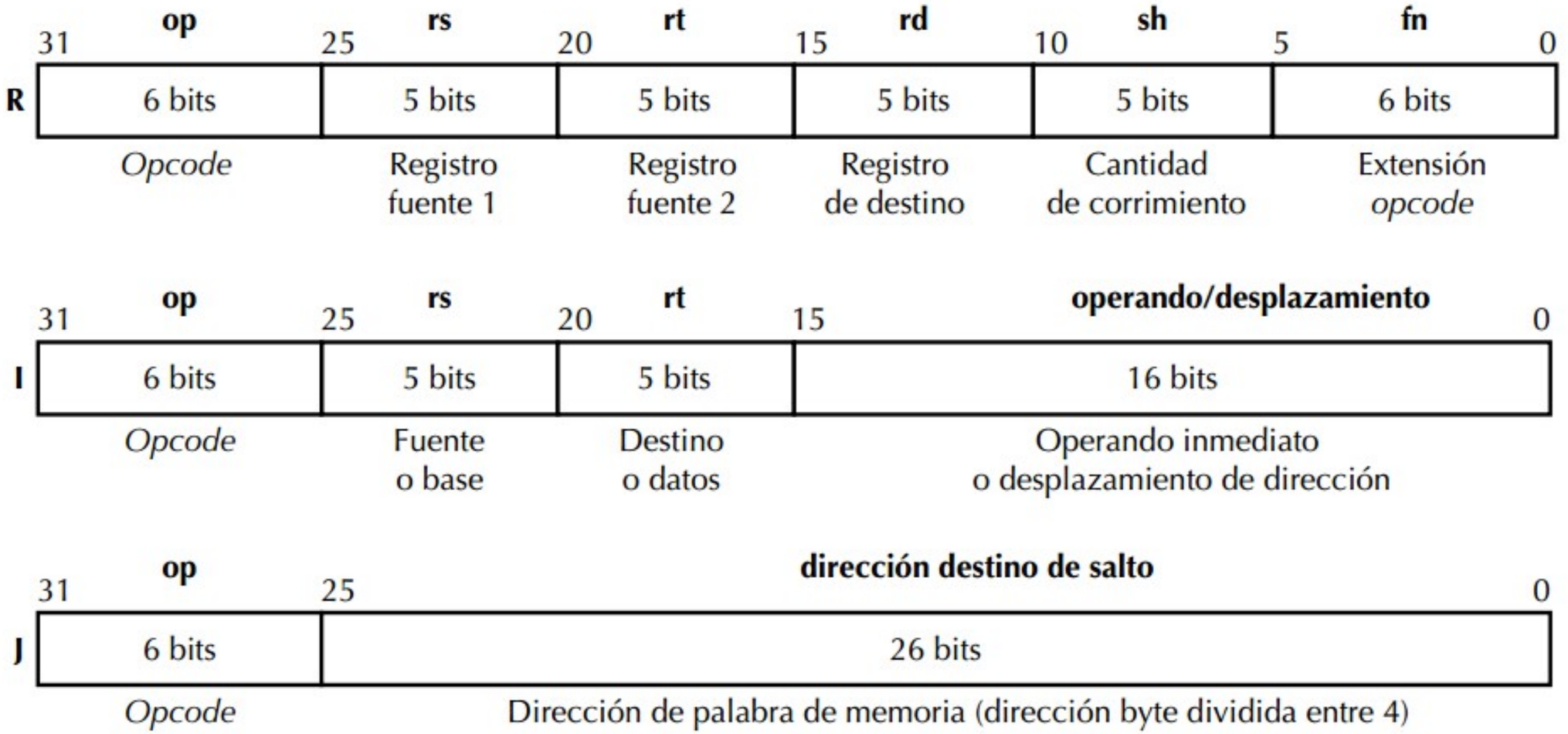
Tipo I

Operaciones con valores inmediatos.

Tipo J

Saltos a direcciones específicas.

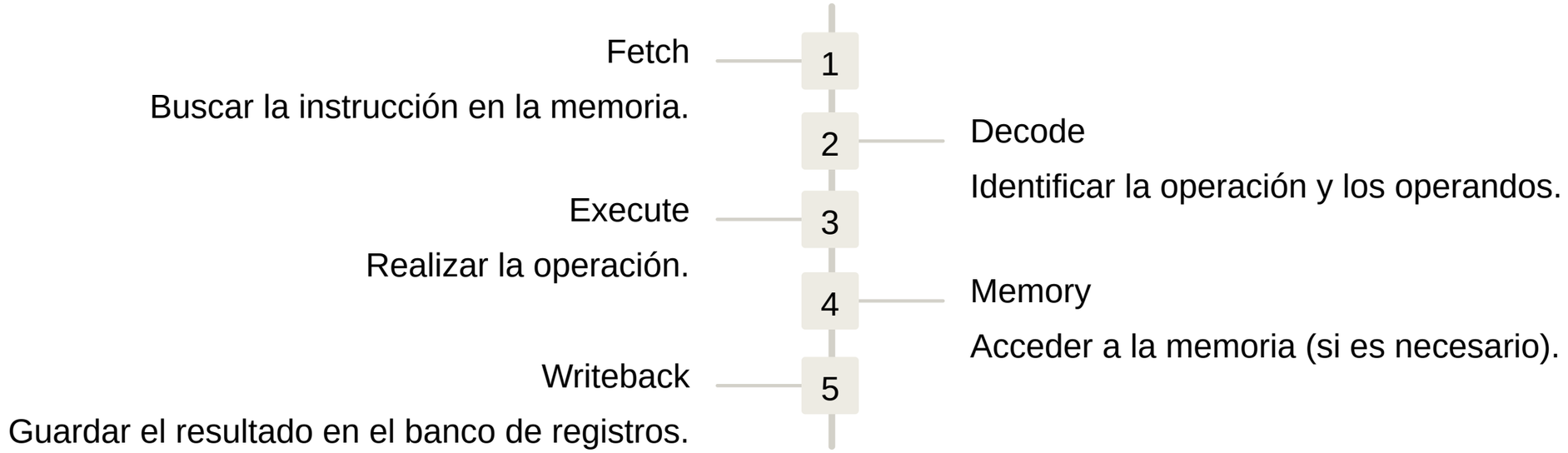
Formato de Instrucciones MIPS



Ciclo de Ejecución de Instrucciones

El ciclo de ejecución de instrucciones es el proceso fundamental que sigue la CPU para ejecutar cada instrucción. Este ciclo se divide en cinco fases principales: Fetch (Búsqueda), Decode (Decodificación), Execute (Ejecución), Memory (Memoria) y Writeback (Escritura de Vuelta).

Cada fase tiene un propósito específico. Fetch busca la instrucción en la memoria. Decode identifica la operación y los operandos. Execute realiza la operación. Memory accede a la memoria si es necesario. Writeback guarda el resultado en el banco de registros. Comprender este ciclo es esencial para optimizar el rendimiento del procesador.



Ciclo de Instrucción en un Procesador de Un Solo Ciclo

1.Fetch (Captura): La CPU obtiene la instrucción desde la memoria utilizando la dirección indicada por el Contador de Programa (PC).

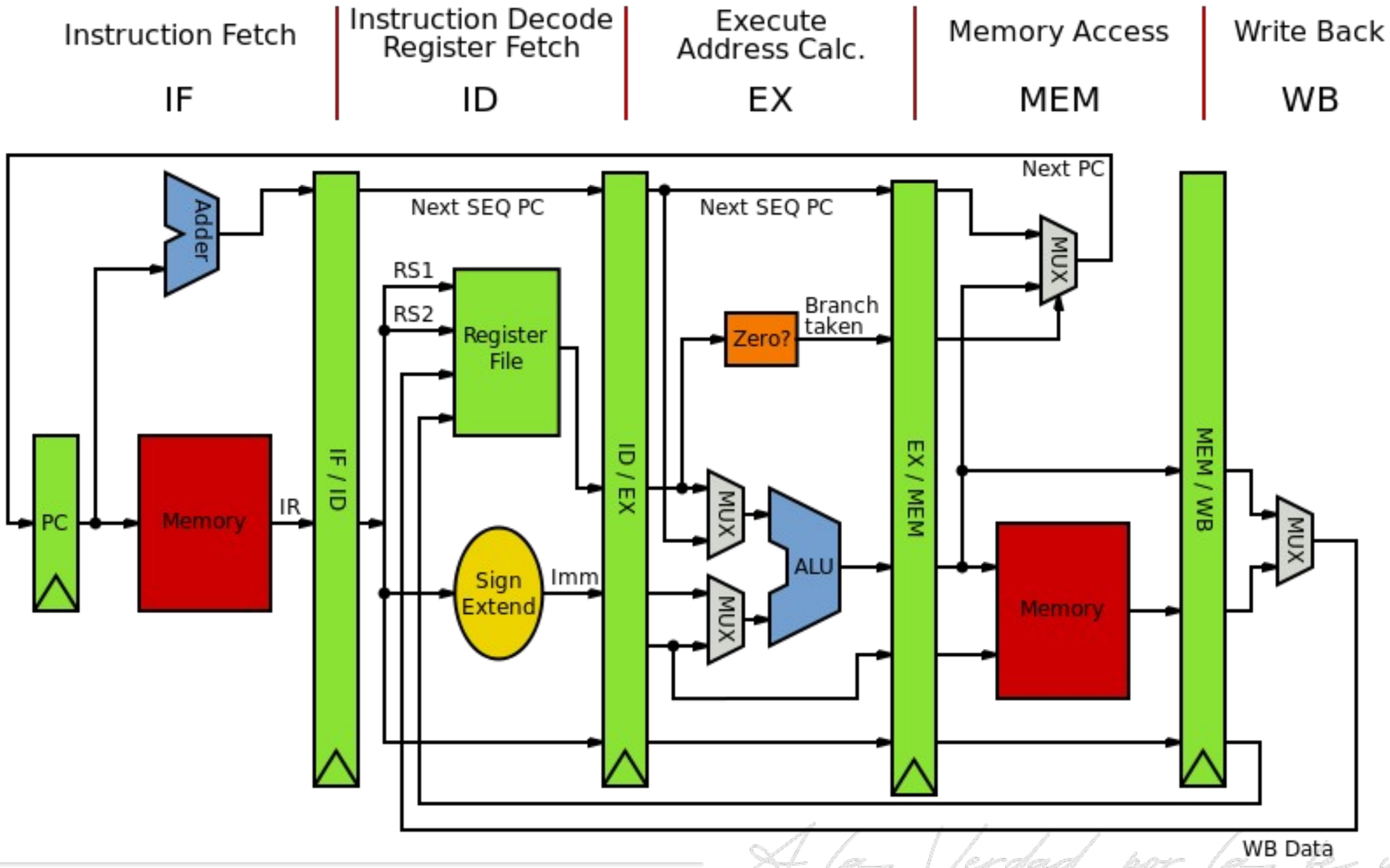
2.Decode (Decodificación): La instrucción capturada se interpreta para determinar qué operación se debe realizar y cuáles son los operandos involucrados.

3.Execute (Ejecución): La Unidad Aritmético-Lógica (ALU) realiza la operación especificada por la instrucción.

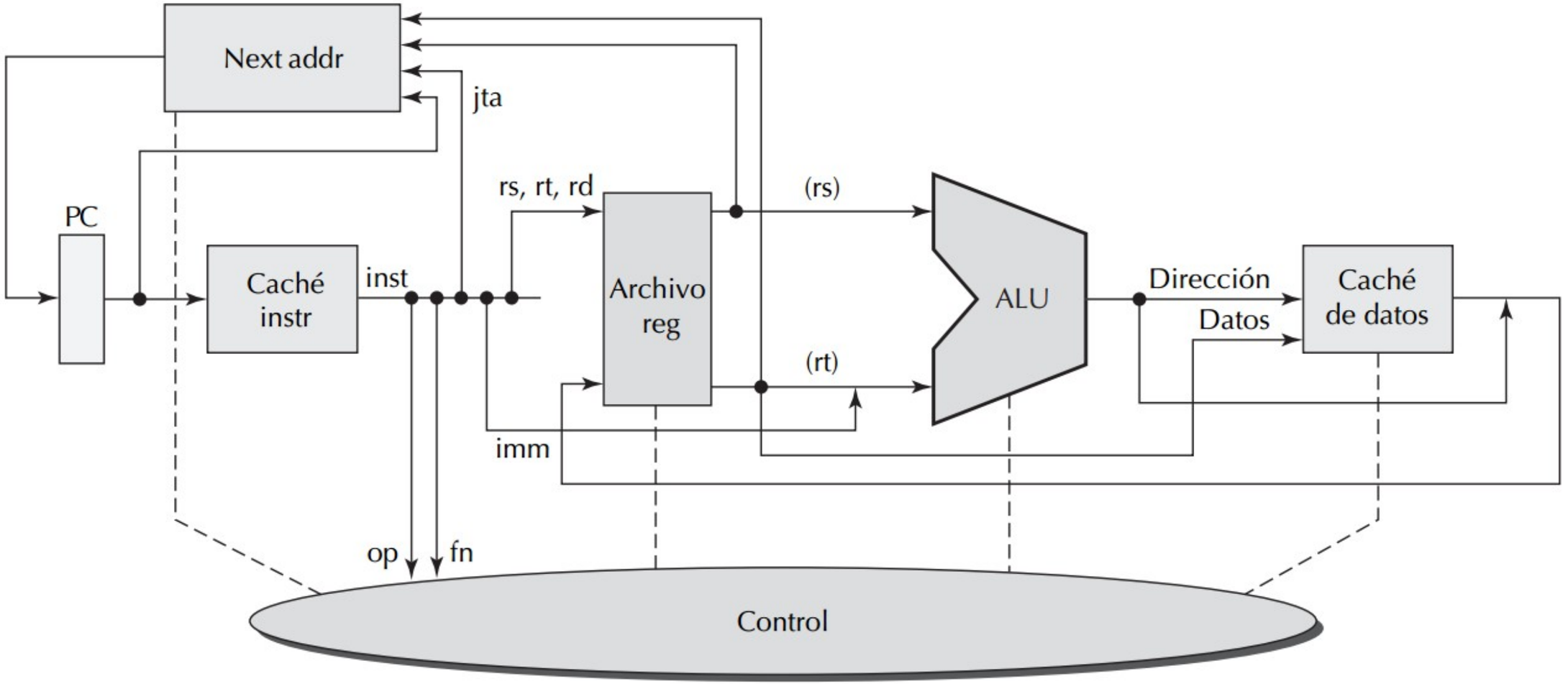
4.Memory Access (Acceso a Memoria): Si la instrucción implica una lectura o escritura en memoria, se realiza en esta etapa.

5.Write-back (Escritura de Resultados): El resultado de la operación se escribe en el registro de destino correspondiente.

Ciclo de Instrucción en un Procesador de Un Solo Ciclo



Ciclo de Instrucción en un Procesador de Un Solo Ciclo



Fase de Fetch

La fase de Fetch es la primera etapa del ciclo de instrucción. Su propósito es obtener la instrucción desde la memoria de instrucciones. Los componentes clave involucrados son el Contador de Programa (PC) y la memoria de instrucciones. El PC contiene la dirección de la siguiente instrucción a ejecutar.

El proceso de lectura implica utilizar el valor del PC para acceder a la memoria de instrucciones y obtener la instrucción correspondiente. Luego, el PC se incrementa para apuntar a la siguiente instrucción en la secuencia. Esta fase es crítica para asegurar que la CPU tenga un flujo constante de instrucciones para ejecutar.

1	2	3
PC	**Memoria de Instrucciones**	**Incremento del PC**
Contiene la dirección de la siguiente instrucción.	Almacena las instrucciones del programa.	Apunta a la siguiente instrucción.

Cache de Instrucciones

La caché de instrucciones es una pequeña memoria de alta velocidad que almacena una copia de las instrucciones más utilizadas recientemente. Su función es reducir el tiempo de acceso a las instrucciones, mejorando así el rendimiento del procesador. Al estar más cerca de la CPU que la memoria principal, la caché permite acceder a las instrucciones más rápidamente.

Las ventajas de usar una caché de instrucciones son evidentes en el rendimiento general del sistema. La estructura básica de la caché incluye líneas de caché, etiquetas y bits de validez. Cuando la CPU necesita una instrucción, primero busca en la caché. Si la instrucción está presente (acierto de caché), se accede a ella rápidamente. Si no está presente (fallo de caché), se busca en la memoria principal y se copia en la caché para futuros accesos.

Alta Velocidad

Acceso rápido a instrucciones.

Mejora el Rendimiento

Reduce el tiempo de acceso a la memoria principal.

Estructura

Líneas de caché, etiquetas y bits de validez.

Decodificación de Instrucciones

La decodificación de instrucciones es el proceso de interpretar la instrucción obtenida en la fase de Fetch. Durante esta fase, la CPU identifica el tipo de instrucción, los operandos involucrados y la operación a realizar. La instrucción se divide en sus campos constituyentes, como el código de operación (opcode), los registros fuente y el registro destino.

A partir de esta decodificación, se generan señales de control que dirigen el funcionamiento de las diferentes unidades del procesador, como la ALU, el banco de registros y la unidad de control de memoria. Estas señales aseguran que cada componente realice la acción correcta en el momento adecuado, garantizando la ejecución precisa de la instrucción.

Interpretación

Identificar el tipo de instrucción y los operandos.

Generación

Crear señales de control para las unidades del procesador.

Precisión

Asegurar la ejecución correcta de la instrucción.

Banco de Registros

El banco de registros es un conjunto de registros de alta velocidad que almacenan datos utilizados frecuentemente por la CPU. En la arquitectura MIPS, el banco de registros consta de 32 registros de propósito general, numerados del $0 al $31. Estos registros son esenciales para realizar operaciones rápidas y eficientes.

Durante la fase de decodificación, se leen los operandos de los registros fuente especificados en la instrucción. En la fase de Writeback, el resultado de la operación se escribe en el registro destino también especificado en la instrucción. El acceso rápido a los registros minimiza la necesidad de acceder a la memoria principal, lo que mejora significativamente el rendimiento.



Almacenamiento Rápido

Datos utilizados frecuentemente por la CPU.



Lectura de Operandos

Acceder a los registros fuente.



Escritura de Resultados

Guardar el resultado en el registro destino.

Unidad Aritmético-Lógica (ALU)

La Unidad Aritmético-Lógica (ALU) es un componente fundamental del procesador que realiza operaciones aritméticas y lógicas. Sus funciones incluyen suma, resta, AND, OR, desplazamiento y comparación. La ALU recibe operandos de los registros fuente y produce un resultado que se puede almacenar en el banco de registros o utilizar en otras fases del ciclo de instrucción.

Las señales de control de la ALU determinan qué operación se va a realizar. Estas señales se generan durante la fase de decodificación y aseguran que la ALU realice la operación correcta según la instrucción en ejecución. La eficiencia de la ALU es crítica para el rendimiento general del procesador.

1

Funciones

Suma, resta, AND, OR, desplazamiento, comparación.

2

Operandos

Recibe datos de los registros fuente.

3

Señales de Control

Determinan la operación a realizar.

Ejecución de Instrucciones Aritméticas

Consideremos un ejemplo de ejecución de una instrucción de suma. Primero, se obtienen los operandos de los registros fuente especificados en la instrucción. Luego, estos operandos se envían a la ALU, que realiza la suma. La señal de control de la ALU se establece para realizar una operación de suma.

El resultado de la suma se almacena temporalmente y luego se escribe en el registro destino durante la fase de Writeback. Este flujo de datos asegura que la operación se realice correctamente y que el resultado esté disponible para futuras operaciones. La precisión y la eficiencia en la ejecución de instrucciones aritméticas son esenciales para el rendimiento del procesador.

1	2	3
Obtener Operandos	**Suma en la ALU**	**Almacenar Resultado**
Leer los registros fuente.	Realizar la operación de suma.	Escribir el resultado en el registro destino.

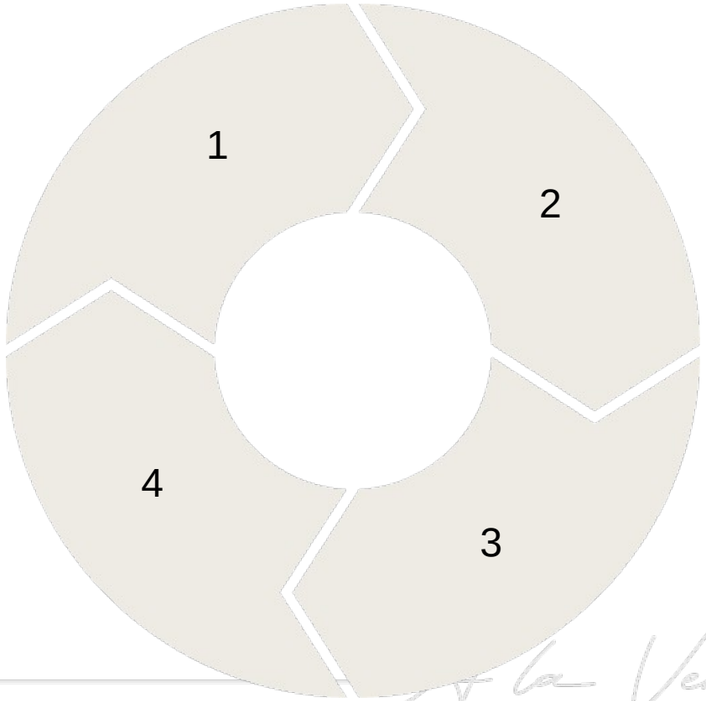
Memoria de Datos

La memoria de datos es donde se almacenan los datos utilizados por el programa en ejecución. Su estructura se organiza en bytes, palabras y bloques, permitiendo el acceso a diferentes tamaños de datos. Las operaciones básicas son lectura (load) y escritura (store), que permiten transferir datos entre la CPU y la memoria.

La alineación de datos es un aspecto crucial para el rendimiento. Los datos deben estar alineados en direcciones de memoria que sean múltiplos de su tamaño. Por ejemplo, una palabra (4 bytes) debe estar alineada en una dirección que sea múltiplo de 4. La falta de alineación puede resultar en accesos más lentos o incluso errores.

Estructura

Organizada en bytes, palabras y bloques.



Lectura (Load)

Transferir datos de la memoria a la CPU.

Alineación

Datos alineados para un acceso eficiente.

Escritura (Store)

Transferir datos de la CPU a la memoria.

Caché de Datos

La caché de datos es similar a la caché de instrucciones, pero almacena datos en lugar de instrucciones. Su función es reducir el tiempo de acceso a los datos, mejorando el rendimiento del procesador. Almacena una copia de los datos más utilizados recientemente, permitiendo un acceso más rápido que la memoria principal.

Existen diferencias clave entre la caché de datos y la caché de instrucciones. La caché de datos necesita políticas de escritura para manejar la coherencia de los datos entre la caché y la memoria principal. Las políticas comunes incluyen write-through (escritura directa) y write-back (escritura diferida). La elección de la política afecta el rendimiento y la complejidad del sistema.

Almacenamiento

Almacena datos de uso frecuente.

Reduce Tiempo

Reduce el tiempo de acceso a los datos.

Políticas de Escritura

Write-through y write-back.

Instrucciones de Carga y Almacenamiento

Las instrucciones de carga (load) y almacenamiento (store) son esenciales para interactuar con la memoria de datos. Las instrucciones de carga transfieren datos desde la memoria a los registros de la CPU, mientras que las instrucciones de almacenamiento transfieren datos desde los registros de la CPU a la memoria.

El cálculo de direcciones efectivas es crucial en estas instrucciones. La dirección efectiva se calcula sumando un desplazamiento (immediate) a un registro base. Esta dirección especifica la ubicación en la memoria donde se leerán o escribirán los datos. La interacción eficiente con la memoria de datos es fundamental para el rendimiento del programa.

1

Load

Transferir datos de la memoria a los registros.

2

Store

Transferir datos de los registros a la memoria.

3

Dirección Efectiva

Calcular la dirección en la memoria.

Unidad de Control

La unidad de control es el cerebro del procesador. Su función principal es generar las señales de control necesarias para coordinar el funcionamiento de las diferentes unidades del procesador. Estas señales controlan la ALU, el banco de registros, la memoria y otras unidades, asegurando que cada componente realice la acción correcta en el momento adecuado.

La temporización de las señales es crítica. La unidad de control debe generar las señales en la secuencia correcta y en el momento preciso para garantizar la correcta ejecución de las instrucciones. La complejidad de la unidad de control aumenta con la complejidad del conjunto de instrucciones y la arquitectura del procesador.

Coordinación

Generar señales para todas las unidades.

Temporización

Asegurar la secuencia correcta de señales.

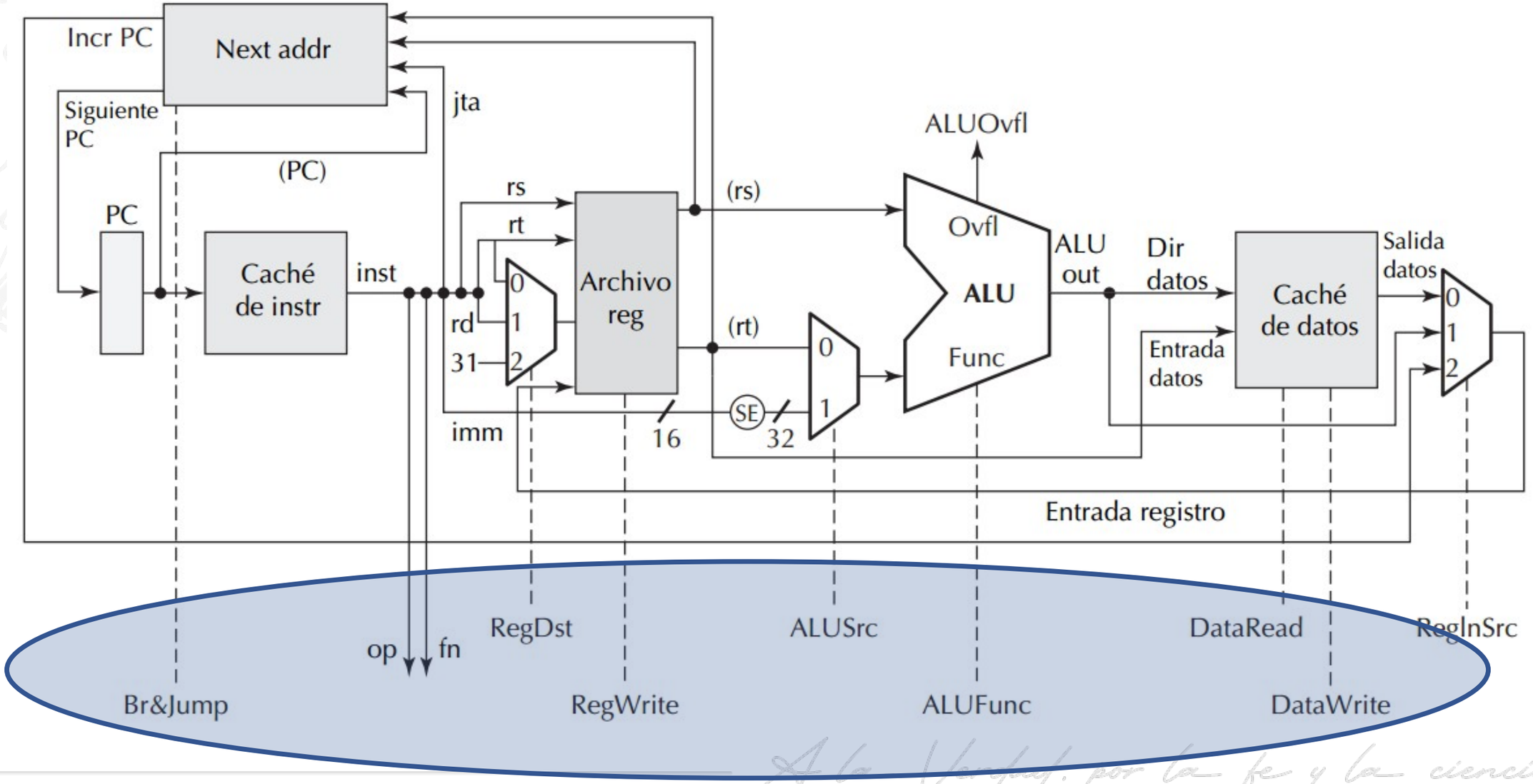
Complejidad

Depende de la arquitectura del procesador.

Unidad de Control



Unidad de Control



Unidad de Control

Instrucción	op	fn	RegWrite	RegDst	RegInSrc	ALUSrc	Add'Sub	LogicFn	FnClass	DataRead	DataWrite	BrType	PCSrc
Carga superior inmediata	001111		1	00	01	1			00	0	0	00	00
Suma	000000	100000	1	01	01	0	0		10	0	0	00	00
Resta	000000	100010	1	01	01	0	1		10	0	0	00	00
Fija menor que	000000	101010	1	01	01	0	1		01	0	0	00	00
Suma inmediato	001000		1	00	01	1	0		10	0	0	00	00
Fija menor que inmediato	001010		1	00	01	1	1		01	0	0	00	00
AND	000000	100100	1	01	01	0		00	11	0	0	00	00
OR	000000	100101	1	01	01	0		01	11	0	0	00	00
XOR	000000	100110	1	01	01	0		10	11	0	0	00	00
NOR	000000	100111	1	01	01	0		11	11	0	0	00	00
AND inmediata	001100		1	00	01	1		00	11	0	0	00	00
OR inmediata	001101		1	00	01	1		01	11	0	0	00	00
XOR inmediata	001110		1	00	01	1		10	11	0	0	00	00
Carga palabra	100011		1	00	00	1	0		10	1	0	00	00
Almacena palabra	101011		0			1	0		10	0	1	00	00
Salto	000010		0							0	0		01
Salta registro	000000	001000	0							0	0		10
Branch en menor que 0	000001		0							0	0	11	00
Branch en igual	000100		0							0	0	01	00
Branch en no igual	000101		0							0	0	10	00
Saltar y ligar	000011		1	10	10					0	0	00	01
Petición de sistema	000000	001100	0							0	0		11

* *Nota*: Las entradas en blanco son del tipo "no importa".

Ruta de Datos

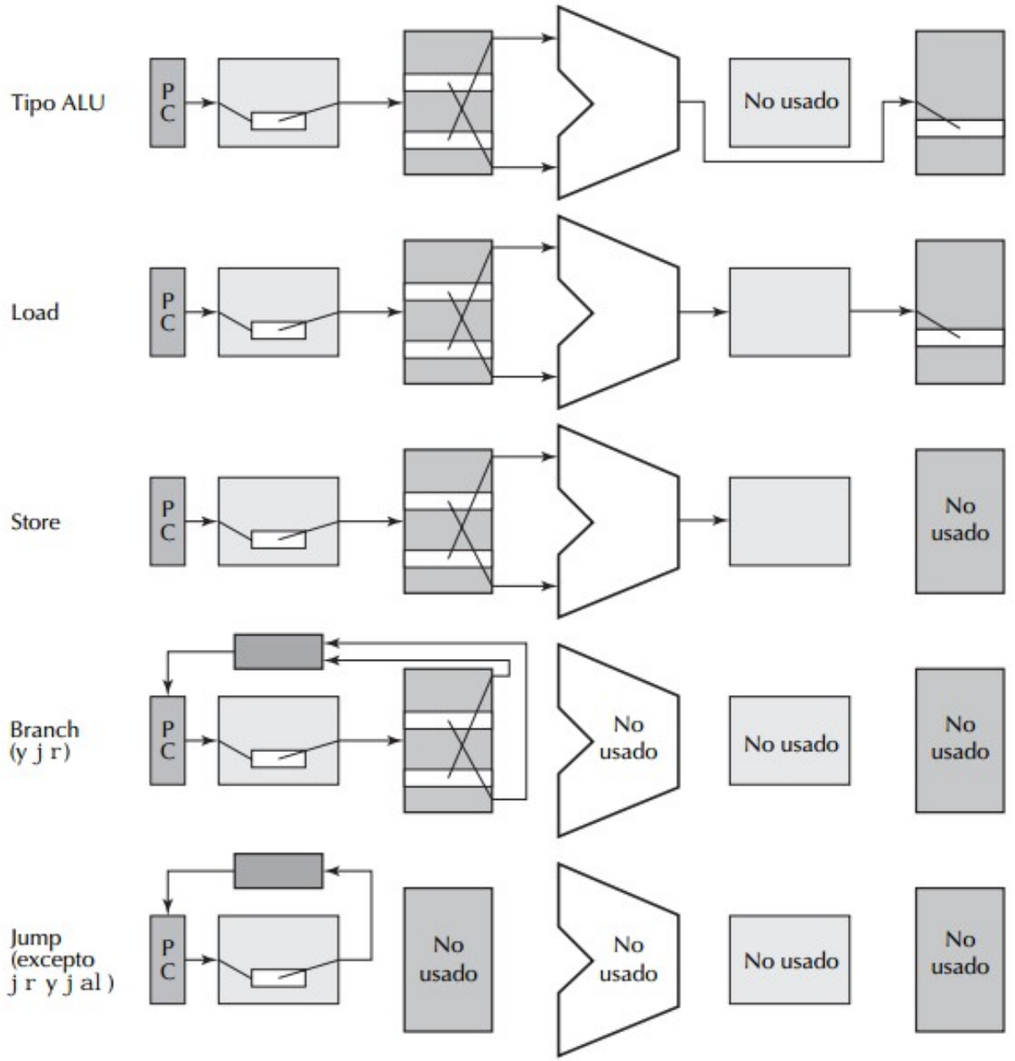
LOAD

Acceso de instrucción	2 ns
Leer registro	1 ns
Operación ALU	2 ns
Acceso a caché de datos	2ns
Escritura de registro	1ns

PERIODO 8nS
VELOCIDAD DE RELOJ 125 MHZ

Tipo R	6 ns No acceso a caché de datos
Load	8 ns Ciclo completo
Store	7 ns No escritura en registro
Branch	5 ns Fetch + lectura registro + formación siguiente dirección
Jump	3 ns Fetch + decodificación instrucción

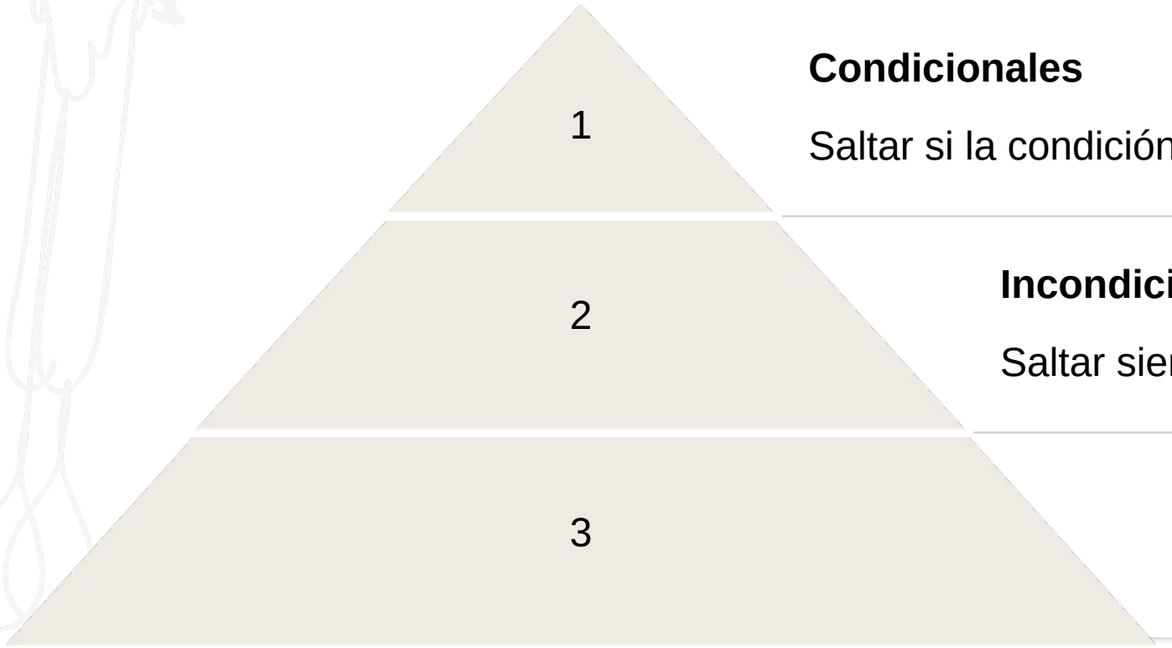
PROMEDIO PONDERADO ~ = 6.36 nS



Instrucciones de Salto

Las instrucciones de salto alteran el flujo de ejecución del programa. Existen dos tipos principales: condicionales e incondicionales. Las instrucciones de salto condicional evalúan una condición y, si es verdadera, saltan a una dirección especificada. Las instrucciones de salto incondicional saltan siempre a la dirección especificada.

El cálculo de la dirección de salto es un proceso fundamental. La dirección se puede calcular sumando un desplazamiento a la dirección actual del PC o utilizando una dirección absoluta. El impacto de las instrucciones de salto en el flujo de ejecución es significativo, permitiendo la implementación de bucles, condicionales y funciones.



Condicionales

Saltar si la condición es verdadera.

Incondicionales

Saltar siempre.

Dirección

Calcular la dirección de salto.

A la Verdad, por la fe y la ciencia

Unidad de Salto

La unidad de salto es responsable de evaluar las condiciones de salto y actualizar el PC con la dirección de salto si la condición es verdadera. Los componentes clave incluyen comparadores, multiplexores y lógica de control. El comparador evalúa la condición, el multiplexor selecciona la dirección de salto o la siguiente dirección secuencial, y la lógica de control coordina la operación.

La actualización del PC en caso de salto es un paso crítico. Si la condición de salto es verdadera, el PC se actualiza con la dirección de salto, alterando el flujo de ejecución del programa. La eficiencia de la unidad de salto es esencial para el rendimiento de programas que utilizan muchas instrucciones de salto.



Comparadores

Evaluar las condiciones de salto.



Multiplexores

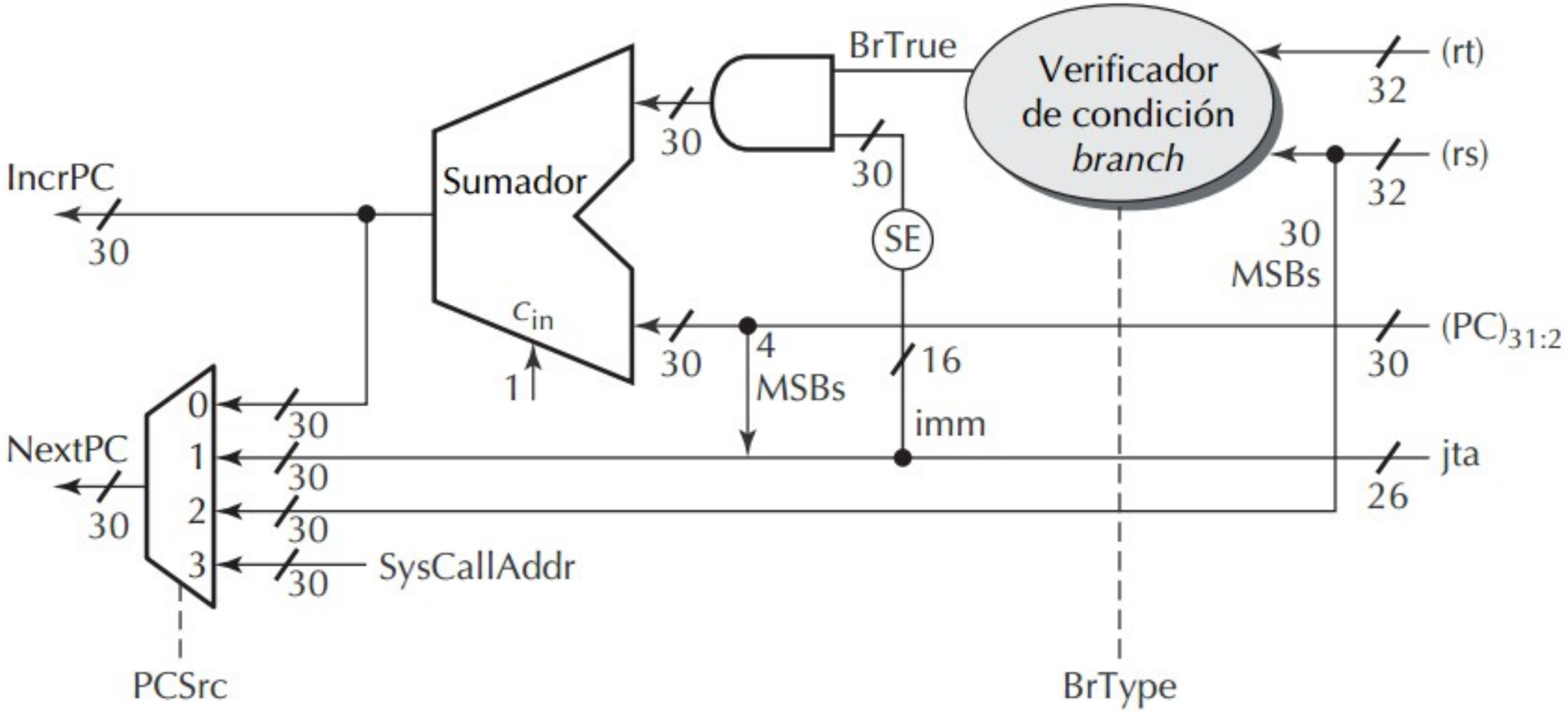
Seleccionar la dirección de salto.



Actualización del PC

Modificar el contador de programa.

Lógica de Dirección Siguiente

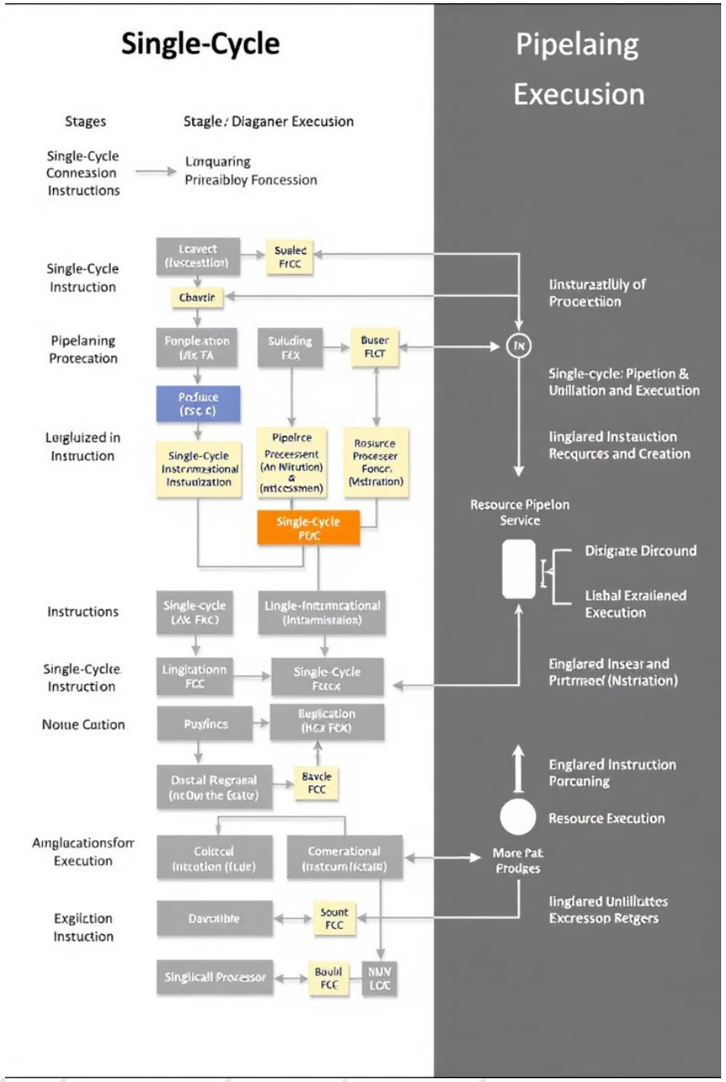


$(PC)_{31:2} + 1$	Opción por defecto
$(PC)_{31:2} + 1 +$ `imm`	Cuando la instrucción es una bifurcación y la condición se satisface
$(PC)_{31:28} \mid$ `jta`	Cuando la instrucción es `j` o `jal`
$($`rs`$)_{31:2}$	Cuando la instrucción es `jr`
SysCallAddr	Dirección de arranque de una rutina de sistema operativo

Pipeline vs. Ciclo Único

En un procesador de ciclo único, cada instrucción se ejecuta completamente antes de que comience la siguiente. Esto simplifica el diseño pero limita el rendimiento. En un procesador pipeline, varias instrucciones se ejecutan simultáneamente en diferentes fases del ciclo de instrucción, lo que aumenta el rendimiento.

Las ventajas del pipeline incluyen mayor rendimiento y utilización de recursos. Las desventajas incluyen mayor complejidad y la necesidad de manejar dependencias y conflictos. Los procesadores pipeline son comunes en sistemas de alto rendimiento, mientras que los procesadores de ciclo único son útiles para la enseñanza y aplicaciones simples.



1

Ciclo Único

Simple pero de bajo rendimiento.

2

Pipeline

Mayor rendimiento pero más complejo.

Optimización del Rendimiento

Existen varias técnicas para mejorar el rendimiento en MIPS. El paralelismo a nivel de instrucción (ILP) busca ejecutar múltiples instrucciones simultáneamente. La predicción de saltos intenta predecir si un salto condicional se tomará o no, permitiendo que la CPU comience a buscar la siguiente instrucción antes de que se conozca el resultado del salto.

Otras técnicas incluyen la optimización del código, el uso eficiente de la caché y la minimización de las dependencias de datos. La optimización del rendimiento es un proceso continuo que requiere una comprensión profunda de la arquitectura del procesador y el comportamiento del programa.

1	**Paralelismo** Ejecutar múltiples instrucciones simultáneamente.
2	**Predicción** Predecir el resultado de los saltos.
3	**Optimización** Mejorar el código y el uso de la caché.

A la Verdad, por la fe y la ciencia

Programación en Ensamblador MIPS

La programación en ensamblador MIPS implica escribir código utilizando las instrucciones MIPS directamente. La sintaxis básica incluye el uso de nemónicos para representar las instrucciones y el uso de registros y direcciones de memoria para manipular datos. El ensamblador MIPS es una herramienta poderosa para entender cómo funciona el procesador a bajo nivel.

Existen diversas herramientas de desarrollo y simulación para MIPS, como MARS y SPIM. Estas herramientas permiten escribir, ensamblar, ejecutar y depurar código MIPS en un entorno simulado. La programación en ensamblador MIPS es una habilidad valiosa para ingenieros y científicos de la computación que deseen comprender y optimizar el rendimiento del software.

1

Sintaxis

Uso de nemónicos y registros.

2

Herramientas

MARS y SPIM para desarrollo y simulación.

3

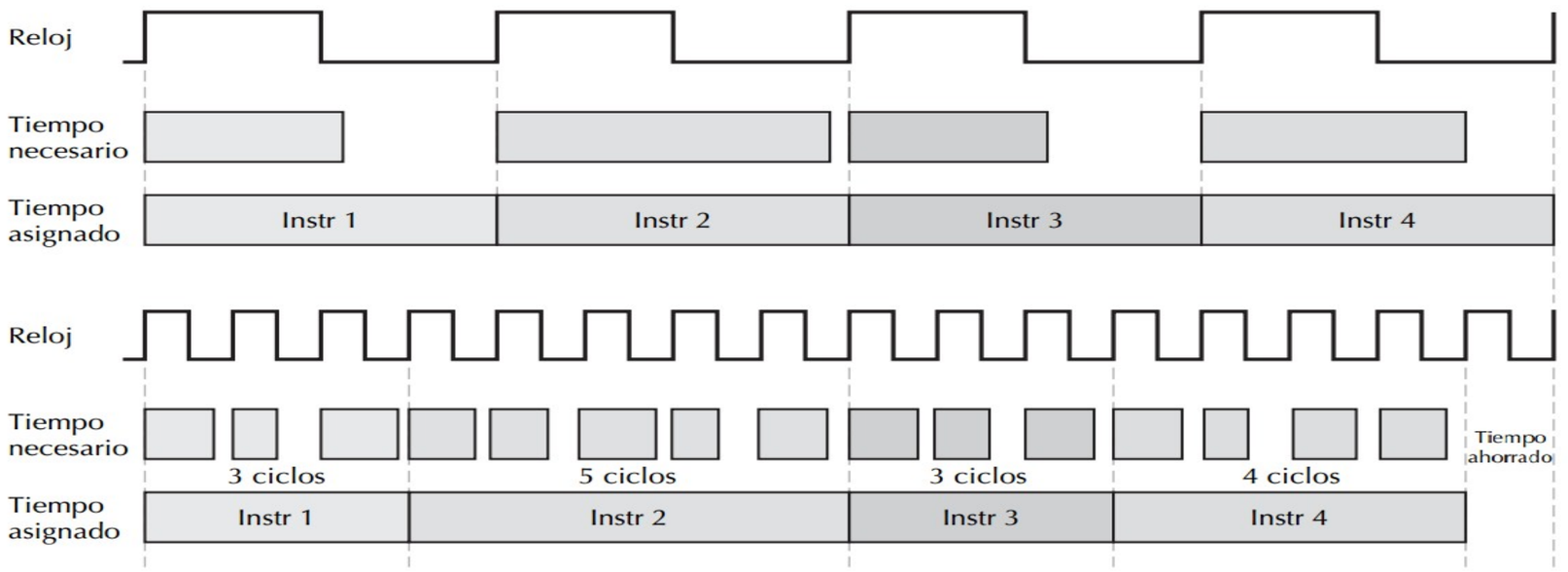
Entendimiento

Comprender el funcionamiento del procesador.

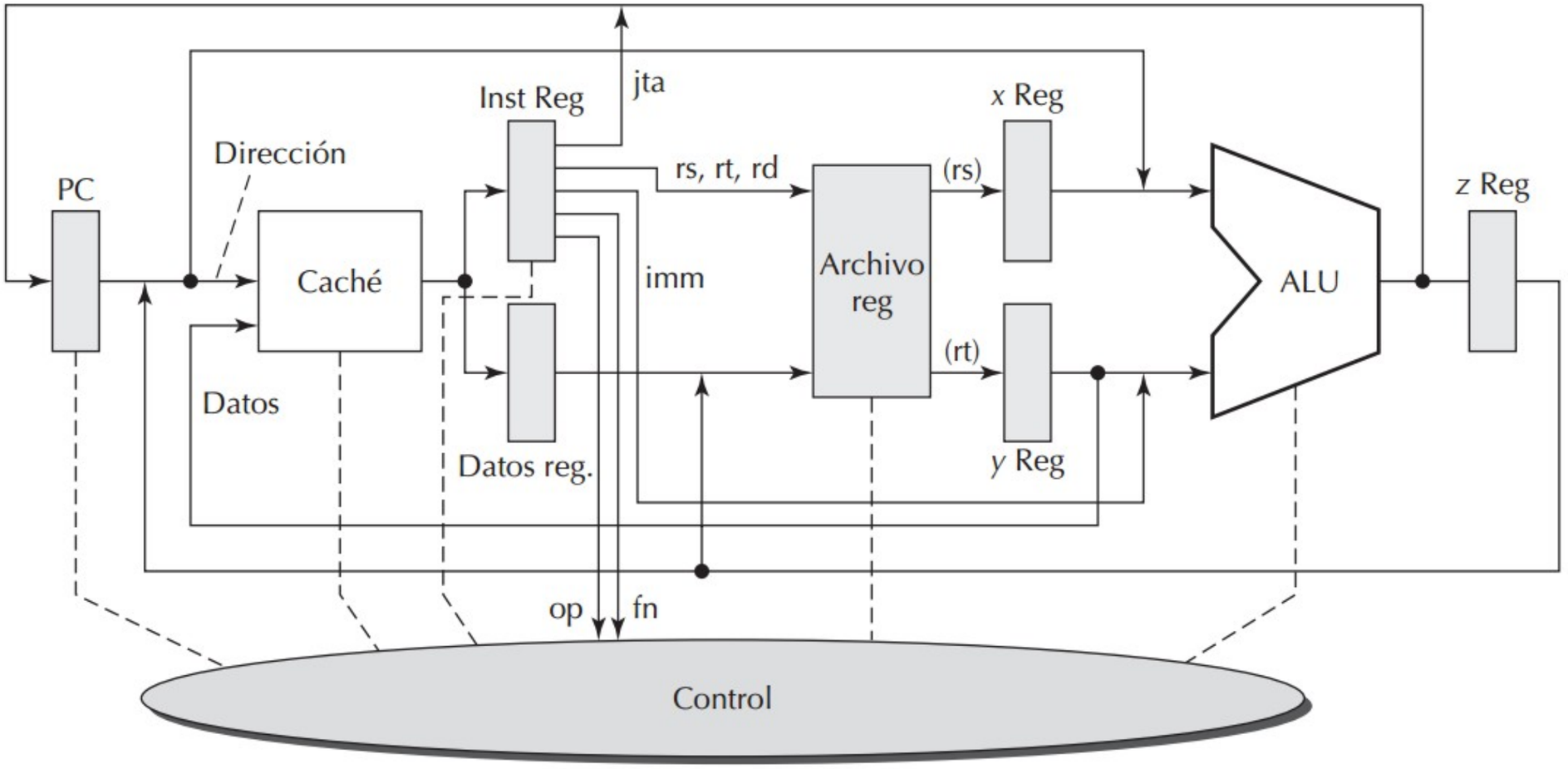
Múltiples Ciclos de Instrucción

Cuando las instrucciones son más variadas en complejidad o cuando algunos recursos se deben usar más de una vez durante la misma instrucción, se necesita una implementación multiciclo.

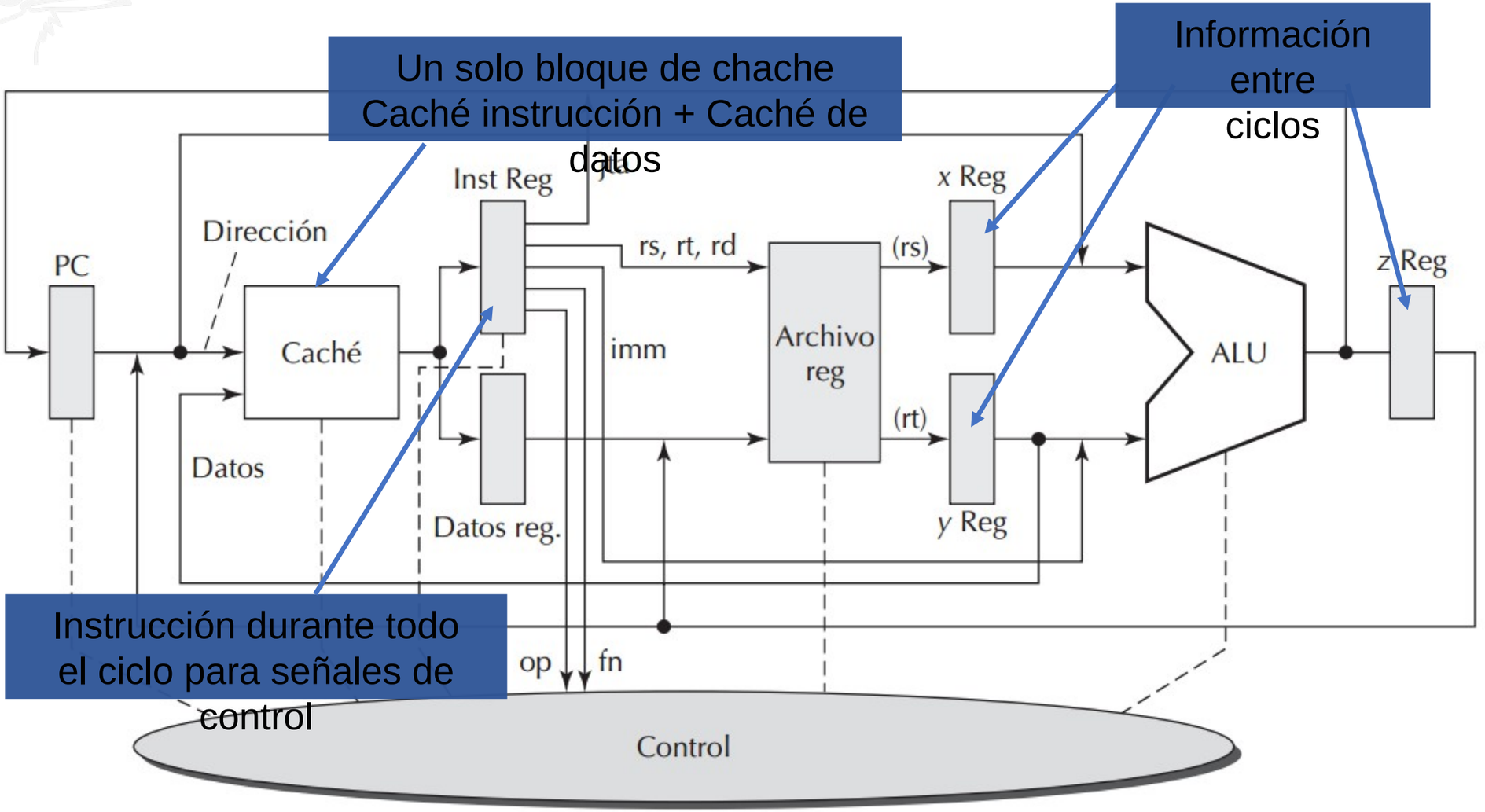
El circuito de control de tal implementación multiciclo es una máquina de estado, con un número de estados (states) para ejecución normal y estados adicionales para manejo de excepciones.



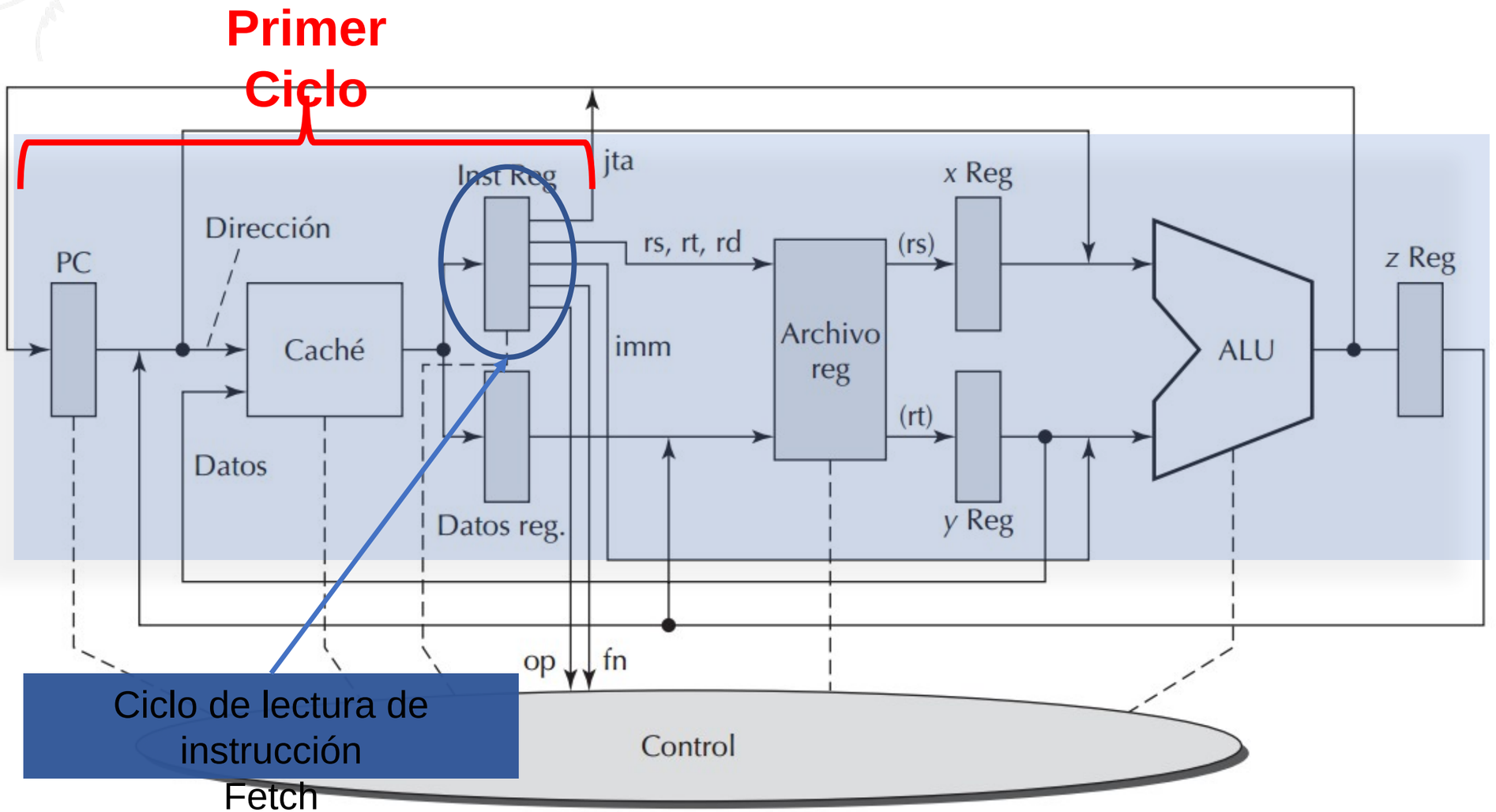
Rutas de Datos de Múltiples Ciclos



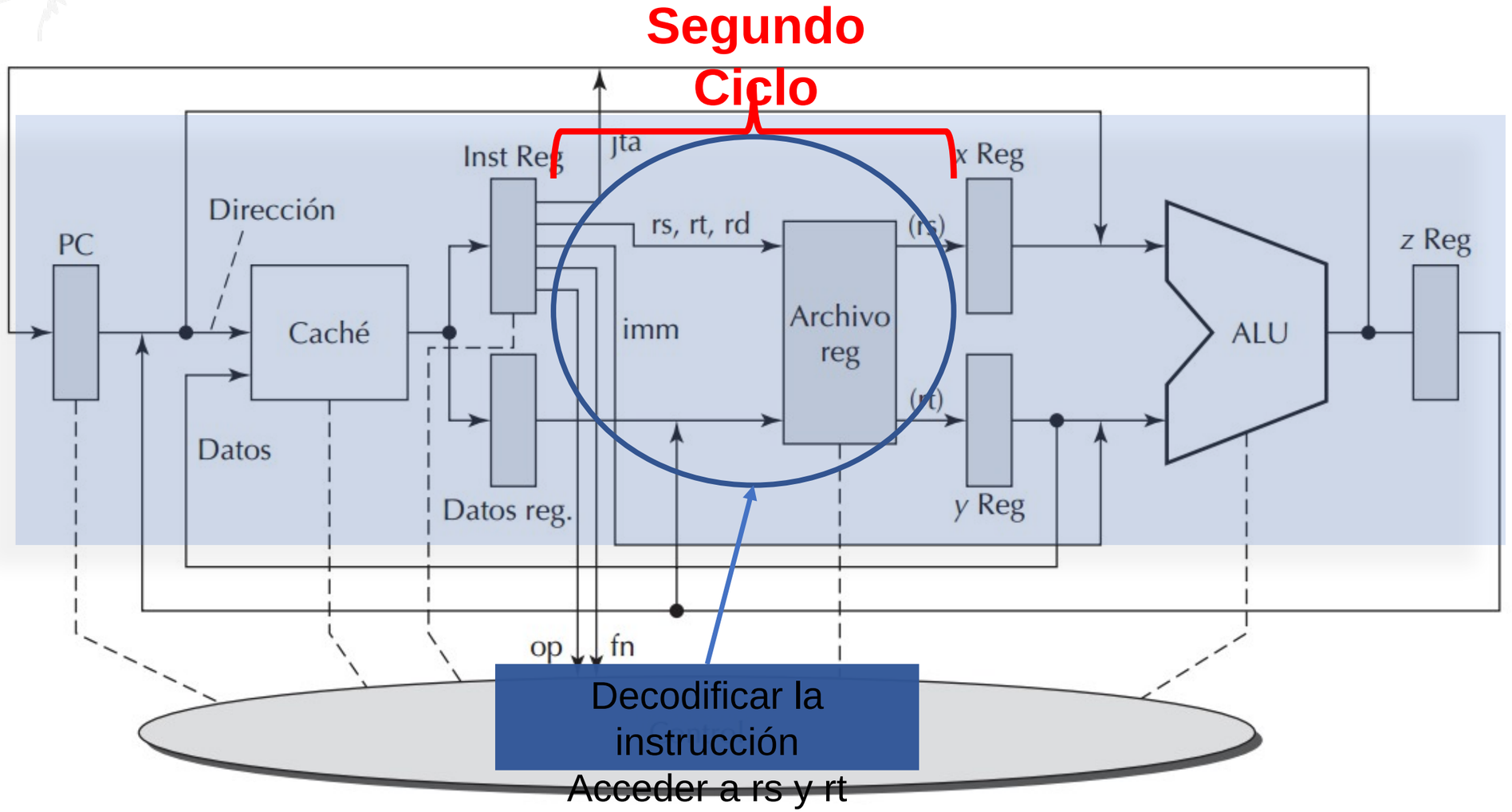
Rutas de Datos de Múltiples Ciclos



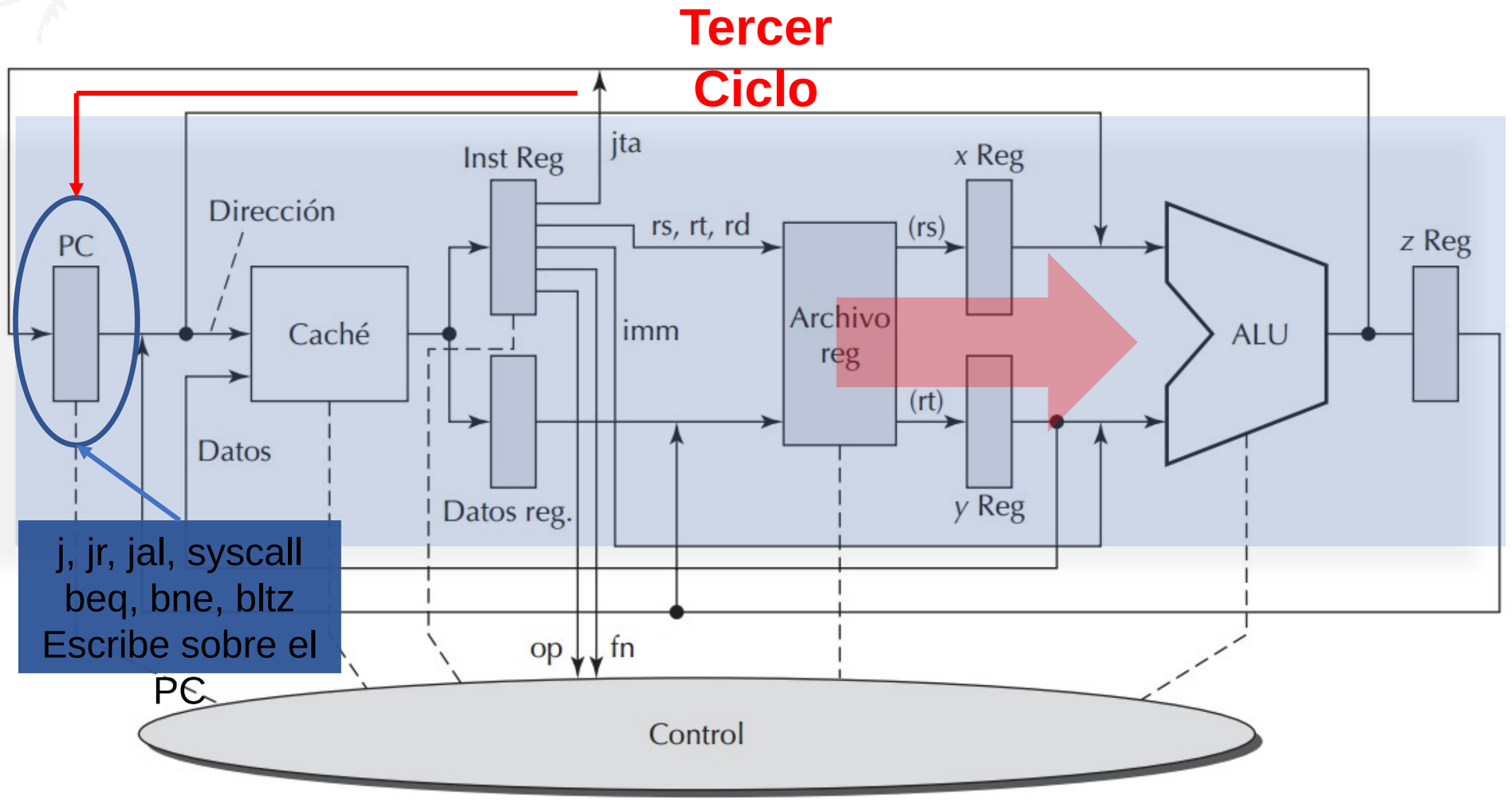
Rutas de Datos de Múltiples Ciclos



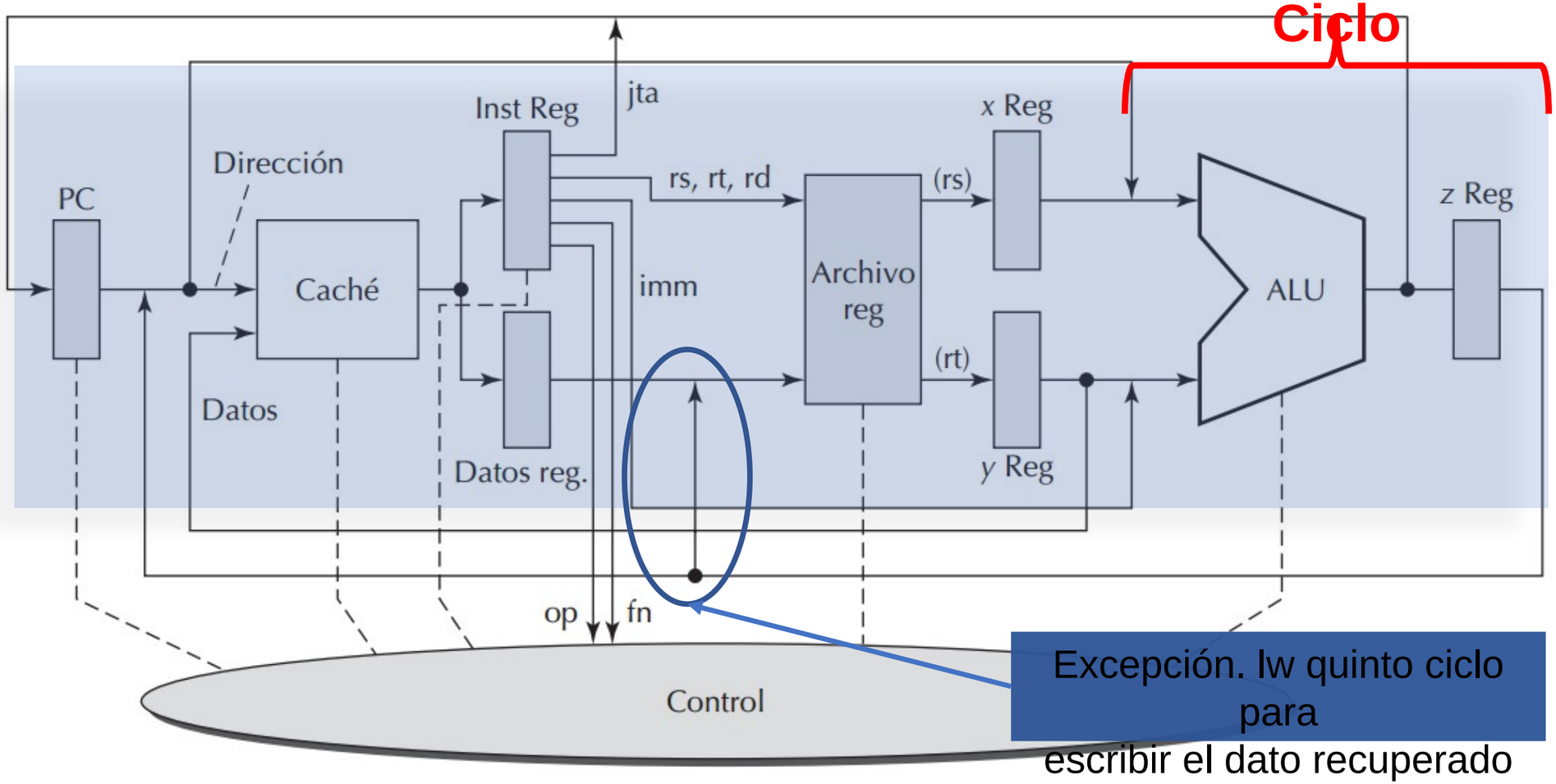
Rutas de Datos de Múltiples Ciclos



Rutas de Datos de Múltiples Ciclos

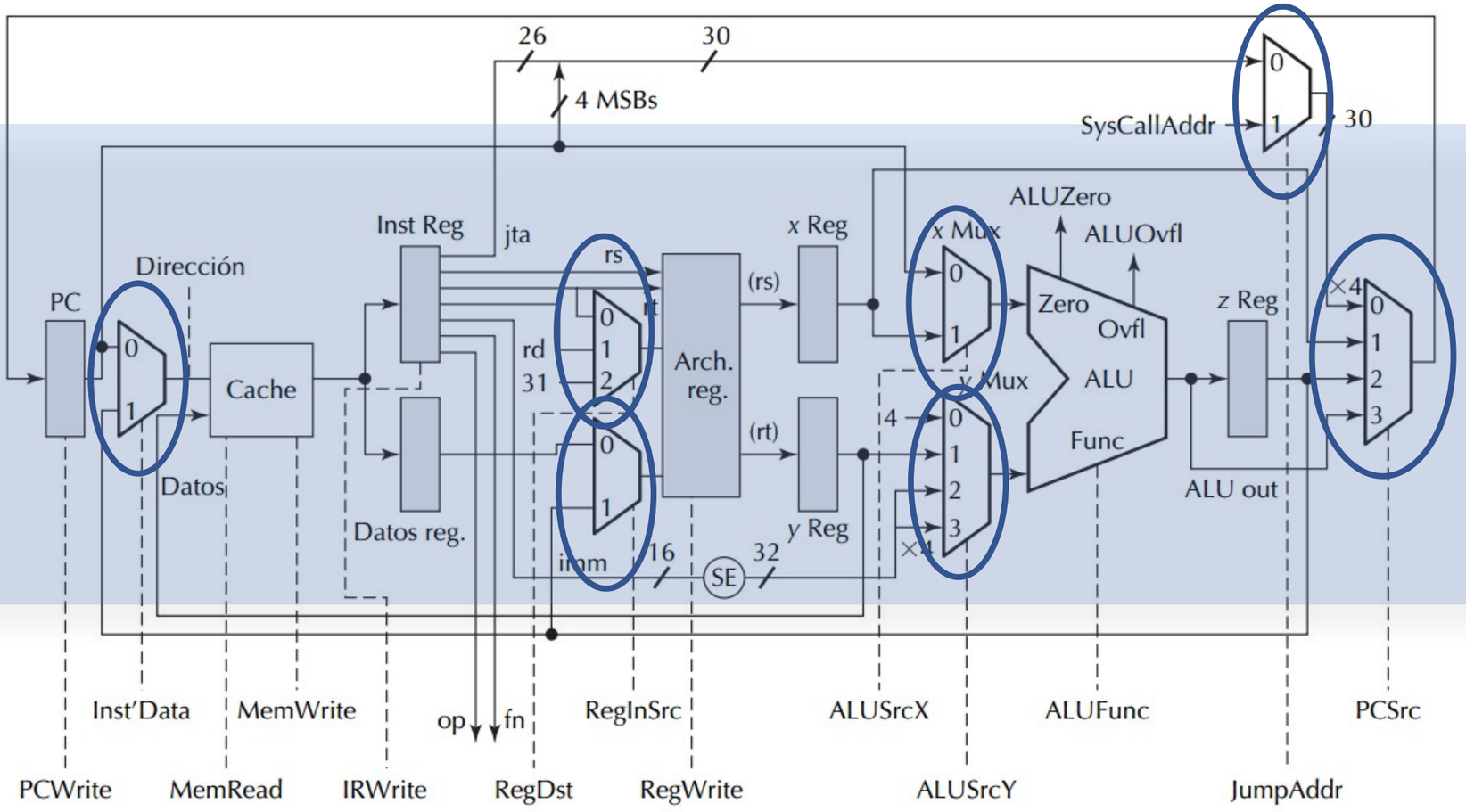


Rutas de Datos de Múltiples Ciclos



Excepción. lw quinto ciclo para escribir el dato recuperado

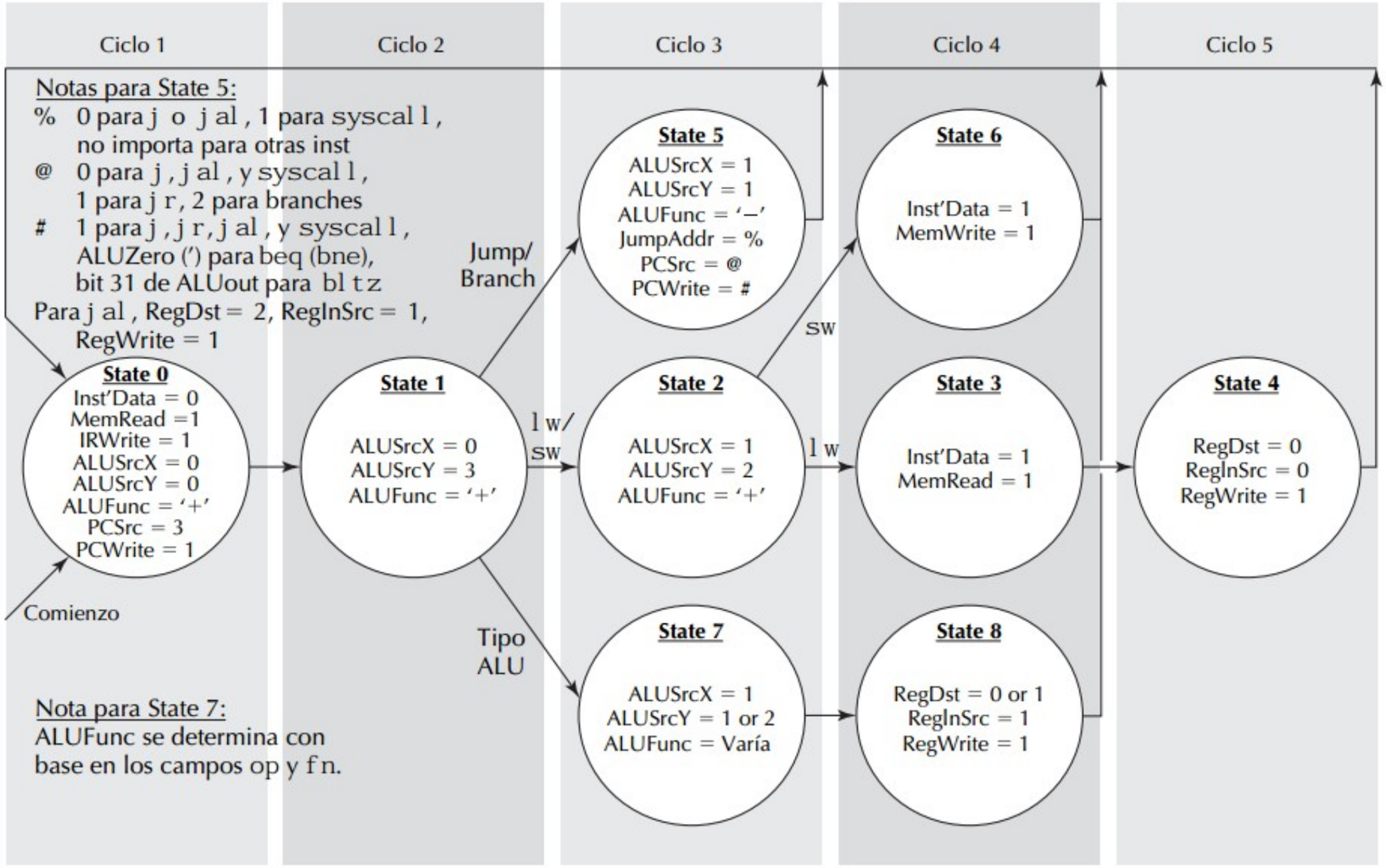
Rutas de Datos de Múltiples Ciclos



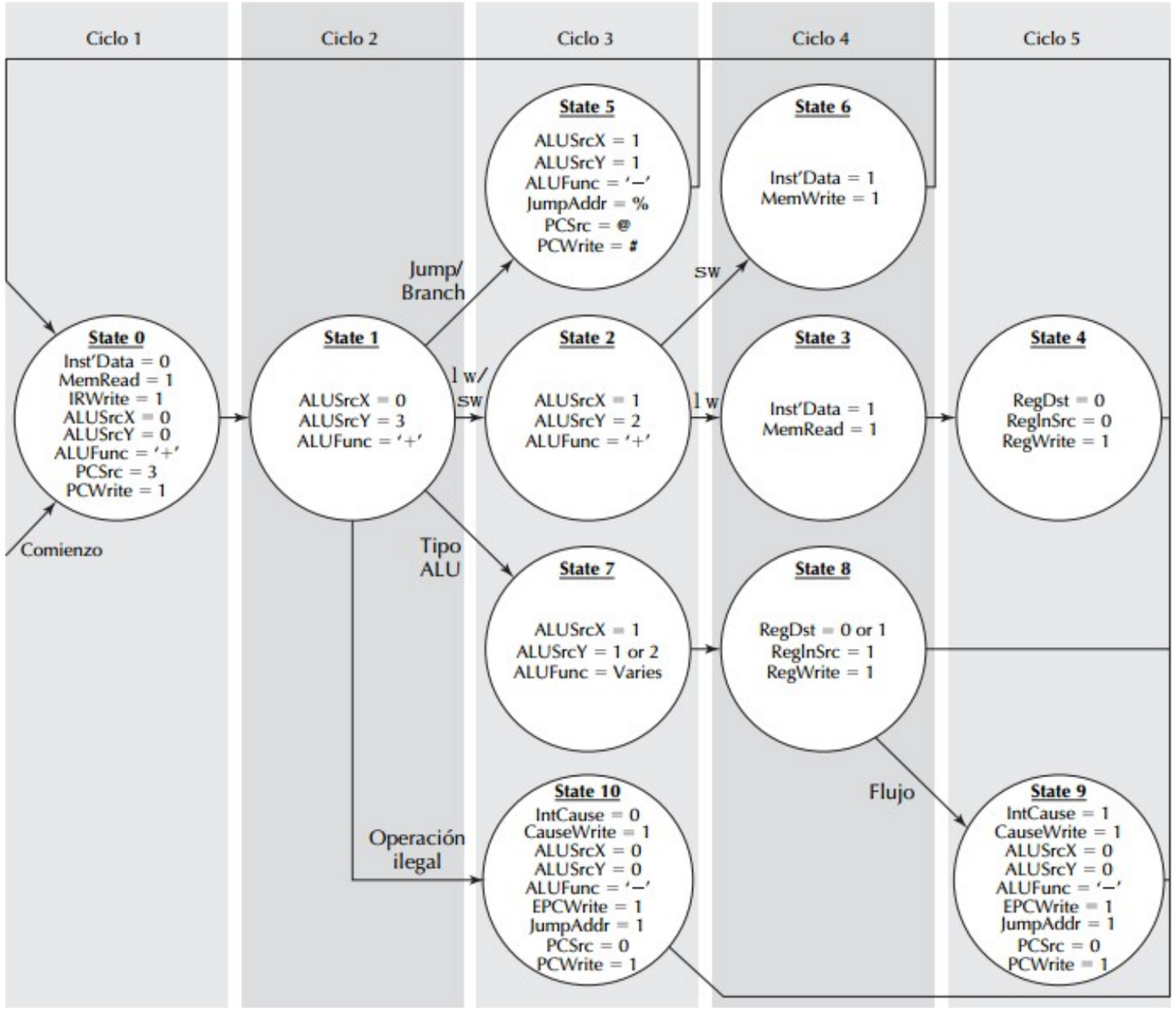
Rutas de Datos de Múltiples Ciclos

Ciclo de reloj	Instrucción	Operaciones	Establecimientos de señal
1 *Fetch* e incremento PC	Cualquiera	Lee la instrucción y la escribe en el registro de instrucción; incrementa PC	Inst′Data = 0, MemRead = 1 IRWrite = 1, ALUSrcX = 0 ALUSrcY = 0, ALUFunc = '+' PCSrc = 3, PCWrite = 1
2 Decodifica y lectura de registro	Cualquiera	Lee `rs` y `rt` en los registros *x* y *y*; calcula la dirección de bifurcación y la salva en el registro *z*	ALUSrcX = 0, ALUSrcY = 3 ALUFunc = '+'
3 Operación ALU y actualización PC	Tipo ALU	Realiza operación ALU y salva el resultado en el registro *z*	ALUSrcX = 1, ALUSrcY = 1 o 2 ALUFunc: Varies
	Load/Store	Suma los valores base y *offset*; salva en registro *z*	ALUSrcX = 1, ALUSrcY = 2 ALUFunc = '+'
	Branch	Si (*x* reg) = ≠ < (*y* reg), fija PC para dirección destino de bifurcación	ALUSrcX = 1, ALUSrcY = 1 ALUFunc = '−', PCSrc = 2 PCWrite = ALUZero o ALUZero′ o ALUOut $_{31}$
	Jump	Fija PC a la dirección blanco `jta`, SysCallAddr o (`rs`)	JumpAddr = 0 o 1 PCSrc = 0 o 1, PCWrite = 1
4 Escribir registro o acceso a memoria	Tipo ALU	Escribe reg *z* en `rd`	RegDst = 1, RegInSrc = 1 RegWrite = 1
	Load	Lee memoria en reg datos	Inst′Data = 1, MemRead = 1
	Store	Copia reg *y* en memoria	Inst′Data = 1, MemWrite = 1
5 Escribir registro para `lw`	Load	Copia registro de datos en `rt`	RegDst = 0, RegInSrc = 0 RegWrite = 1

Maquinas de Control de Estado



Excepciones



UCO
Universidad Católica de Oriente
Vigilada Mineducación

Renovación Acreditación Institucional en ALTA CALIDAD



¡Gracias!